



Universidad Nacional
Autónoma de México



Proyecto Parcial 1

Desarrollo de una FFT de 1024 puntos mediante
mariposas

Asignatura:

Procesamiento Digital de Señales

Profesor:

Dr. Oscar Francisco Fuentes Casarrubias

Grupo: 03

Integrantes Equipo 2:

Espinoza Matamoros Percival Ulises

García Cortés Adolfo de Jesús

Lugo Manzano Rodrigo

Fecha de entrega:

07 de Octubre del 2026

Semestre: 2027 - 1



Índice

1. Introducción	1
2. Planteamiento del problema	2
3. Desarrollo por mariposas	4
4. Estructura de 10 etapas	5
5. Ordenamiento de datos	7
6. Diagrama o esquema	9
7. Complejidad Computacional	10
8. Conclusiones	14
Referencias	14



Objetivo

El objetivo de (Esta actividad es que el estudiante comprenda y documente la estructura de una Transformada Rápida de Fourier (FFT) de 1024 puntos, desarrollada mediante CI uso de operaciones tipo mariposa. Se espera que el trabajo muestre una comprensión clara de la descomposición algorítmica, la organización por etapas y el papel de los factores de giro (*twiddle factors*) en el cálculo eficiente de la DFT.

1. Introducción

La Transformada Discreta de Fourier (DFT) es una herramienta fundamental en el procesamiento digital de señales, utilizada para analizar la frecuencia de señales discretas. Para implementar el algoritmo de la DFT es mediante multiplicación de matrices, lo que resulta en una complejidad computacional de $O(N^2)$. Sin embargo, la Transformada Rápida de Fourier (FFT) es un algoritmo eficiente que reduce esta complejidad a $O(N \log N)$ mediante la descomposición de la DFT en etapas más pequeñas utilizando operaciones tipo mariposa.

La FFT reduce significativamente el tiempo de cálculo, esto debido a que aprovecha la simetría y periodicidad de los factores de giro (*twiddle factors*), permitiendo un procesamiento más rápido y eficiente de señales. De manera matemática se puede expresar visualizar como:

Considerando la expresión de la DFT para una señal discreta $x[n]$ de longitud N :

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N-1$$

Donde:

$$W_N = e^{-j \frac{2\pi}{N}}$$

La DFT puede ser reescrita como:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}$$

Esta se puede descomponer en dos sumas, una para los índices pares y otra para los impares:

$$X[k] = \sum_{n=0}^{N/2-1} x[2n] W_N^{k(2n)} + \sum_{n=0}^{N/2-1} x[2n+1] W_N^{k(2n+1)}$$



$$X[k] = \sum_{n=0}^{N/2-1} x[2r](W_N^2)^{rk} + W_N^k \sum_{n=0}^{N/2-1} x[2r+1](W_N^2)^{rk}$$

Sustituyendo la siguiente expresión:

$$W_N^n = e^{-j\frac{2\pi}{N}n} = e^{-j\frac{2\pi}{N/n}}$$

Donde si $n = 2$

$$W_N^2 = e^{-j\frac{2\pi}{N/2}} = W_{N/2}$$

Sustituyendo en la ecuación anterior:

$$X[k] = \sum_{n=0}^{N/2-1} x[2r]W_{N/2}^{rk} + W_N^k \sum_{n=0}^{N/2-1} x[2r+1]W_{N/2}^{rk}$$

Definiendo:

$$G[k] = \sum_{n=0}^{N/2-1} x[2r]W_{N/2}^{rk}$$
$$H[k] = \sum_{n=0}^{N/2-1} x[2r+1]W_{N/2}^{rk}$$

Se puede reescribir la DFT como:

$$X[k] = G[k] + W_N^k H[k] \quad 0 \leq k \leq N/2 - 1$$

$$X[k] = G[k] - W_N^k H[k] \quad N/2 \leq k \leq N - 1$$

A partir de esta expresión, se puede observar que la DFT de tamaño N se puede calcular utilizando dos DFTs de tamaño $N/2$, lo que permite una reducción significativa en la cantidad de cálculos necesarios. Este proceso se repite recursivamente hasta llegar a DFTs de tamaño 2.

2. Planteamiento del problema

Para plantear el problema, se requiere desarrollar una FFT de 1024 puntos utilizando operaciones tipo mariposa. Sin embargo un algoritmo debe ser general, es decir, que pueda ser escalable a cualquier tamaño de entrada N que sea una potencia de 2. Es por eso que para el planteamiento del problema se considera una señal de entrada $x[n]$ de longitud $N = 4$,



una vez deducida la lógica de la FFT para $N = 4$, se puede escalar a $N = 1024$ mediante la repetición del proceso de descomposición en etapas.

El algoritmo de la FFT consiste en los siguientes pasos:

1. Dividir la señal de entrada en dos partes: una con los índices pares y otra con los índices impares.
2. Calcular la DFT de cada una de estas partes utilizando la misma técnica de descomposición recursiva.
3. Combinar los resultados de las DFTs de las partes pares e impares utilizando los factores de giro (*twiddle factors*) para obtener la DFT completa de la señal de entrada.

Antes de plantear el algoritmo, es importante considerar que aunque dada la naturaleza el problema, que se tiene un caso base y es necesario aplicar la descomposición recursiva hasta llegar a este caso base. La implementación de la FFT se puede realizar mediante un enfoque iterativo o recursivo, sin embargo dado que la FFT busca resolver el problema de manera eficiente, se opta por un enfoque iterativo que permite un mejor control sobre la memoria y el flujo de datos.

Definición de la DFT de N puntos:

El proyecto consiste en desarrollar una FFT de 1024 puntos, sin embargo para poder entender la lógica de la FFT se considera una señal de entrada $x[n]$ de longitud $N = 4$.

La condición de este algoritmo es que el tamaño de la señal de entrada N debe ser una potencia de 2, es decir, $N = 2^m$ para algún entero m . En este caso, $N = 4$ cumple con esta condición.

A partir de esta condición se puede conocer el número de etapas necesarias para realizar la FFT, que se calcula como $m = \log_2(N)$. Para $N = 4$, se tiene $m = \log_2(4) = 2$, lo que indica que se requieren 2 etapas para completar la FFT.

Otro factor a considerar es que en cada etapa se realizan $N/2$ operaciones tipo mariposa en total. Estas operaciones se distribuyen en bloques cuyo tamaño aumenta en cada etapa. Para $N = 4$, se realizan $4/2 = 2$ mariposas por etapa: en la primera etapa se tienen dos bloques de tamaño 2, mientras que en la segunda se tiene un bloque de tamaño 4.

De forma que se puede llegar a la conclusión de que para una FFT de N puntos, el número de etapas es $\log_2(N)$ y en cada etapa se realizan $N/2$ operaciones tipo mariposa, distribuidas en bloques cuyo tamaño aumenta en cada etapa.

Considerando un DFT de $N = 1024$ puntos, se conoce que:

- Número de etapas: $m = \log_2(1024) = 10$
- Número de operaciones tipo mariposa por etapa: $1024/2 = 512$

■ Tamaño de los bloques en cada etapa:

- Etapa 1: Bloques de tamaño 2
- Etapa 2: Bloques de tamaño 4
- Etapa 3: Bloques de tamaño 8
- Etapa 4: Bloques de tamaño 16
- Etapa 5: Bloques de tamaño 32
- Etapa 6: Bloques de tamaño 64
- Etapa 7: Bloques de tamaño 128
- Etapa 8: Bloques de tamaño 256
- Etapa 9: Bloques de tamaño 512
- Etapa 10: Bloques de tamaño 1024

3. Desarrollo por mariposas

Una mariposa es una operación que combina dos valores de entrada para producir dos valores de salida, utilizando los factores de giro (*twiddle factors*) para realizar la combinación. La operación se puede representar por medio del siguiente diagrama:

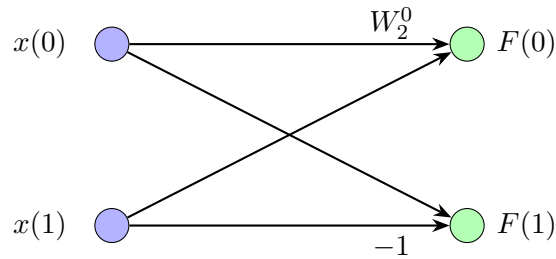


Figura 1: Diagrama de una mariposa de dos puntos.

El diagrama muestra cómo se combinan las entradas $x(0)$ y $x(1)$ para producir las salidas $F(0)$ y $F(1)$. Para evitar una confusión importante, el símbolo -1 que aparece en la rama inferior no representa un segundo factor de giro. Representa el signo negativo de la resta utilizada para calcular la segunda salida.

El factor de giro de una FFT de tamaño N se define como:

$$W_N^k = e^{-j\frac{2\pi}{N}k}$$



donde N es el tamaño de la subtransformada y k identifica el factor de giro utilizado en la mariposa. En una operación tipo mariposa, el factor de giro multiplica a la entrada de la rama inferior antes de combinarla con la entrada superior. Si se define

$$T = W_N^k x(1),$$

las dos salidas se calculan como:

$$F(0) = x(0) + T = x(0) + W_N^k x(1)$$

$$F(1) = x(0) - T = x(0) - W_N^k x(1)$$

Para la mariposa de dos puntos se tiene $N = 2$ y $k = 0$. Por tanto, el factor de giro es:

$$W_2^0 = e^{-j\frac{2\pi}{2}(0)} = e^0 = 1$$

Al sustituir este valor en las ecuaciones anteriores, se obtiene:

$$F(0) = x(0) + W_2^0 x(1) = x(0) + x(1)$$

$$F(1) = x(0) - W_2^0 x(1) = x(0) - x(1)$$

Por lo tanto, en este caso particular el factor de giro no modifica la amplitud ni la fase de $x(1)$, porque vale 1. La diferencia entre ambas salidas proviene exclusivamente de la suma en $F(0)$ y de la resta en $F(1)$. En etapas posteriores, el valor de W_N^k puede ser complejo y entonces sí modifica la fase de la rama inferior antes de realizar ambas operaciones.

4. Estructura de 10 etapas

Una vez definida la operación tipo mariposa, recordando los datos que se puede obtener a partir del número de muestras

- Número de etapas: $m = \log_2(N)$
- Número de operaciones tipo mariposa por etapa: $N/2$
- Tamaño de los bloques en cada etapa: 2^i para $i = 1, 2, \dots, m$

Considerando el caso de una señal de entrada $x[n]$ de longitud $N = 4$, se tiene que el número de etapas es $m = \log_2(4) = 2$. Por lo tanto, se requieren 2 etapas para completar la FFT. En cada etapa se realizan $N/2 = 4/2 = 2$ operaciones tipo mariposa, distribuidas en bloques cuyo tamaño aumenta en cada etapa. Esto se puede ver en el siguiente diagrama:

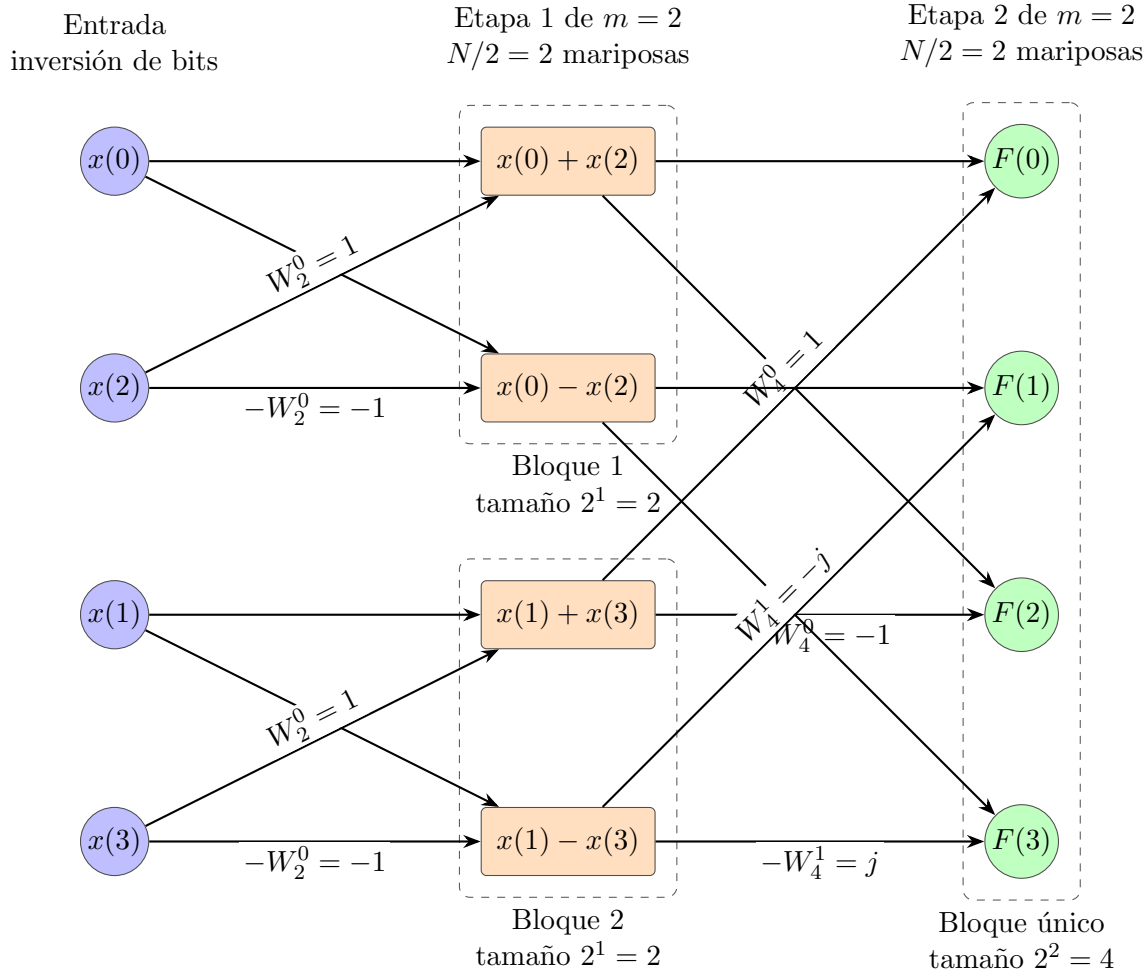


Figura 2: Etapas de una FFT de cuatro puntos utilizando la operación tipo mariposa.

En la primera etapa se forman dos mariposas independientes: una combina $x(0)$ con $x(2)$, y la otra combina $x(1)$ con $x(3)$. Como el tamaño de cada bloque es 2, ambas utilizan $W_2^0 = 1$. En la segunda etapa se combinan los resultados de ambas mariposas. La primera utiliza $W_4^0 = 1$, mientras que la segunda utiliza $W_4^1 = -j$. En cada caso, la rama inferior se multiplica por su factor de giro y después se calculan la suma y la resta.

A través del diagrama se puede identificar cómo cambia la distancia entre las posiciones que se combinan en cada etapa. En la implementación iterativa de la FFT, después de realizar la inversión de bits, esta distancia no se reduce, sino que se duplica en cada etapa. Para la etapa s , la distancia entre las posiciones de una mariposa es:

$$d_s = 2^{s-1}$$



Para $N = 4$, la entrada se procesa en el orden $x(0), x(2), x(1), x(3)$. En la primera etapa, las mariposas combinan posiciones adyacentes del arreglo reordenado, por lo que la distancia es $d_1 = 1$: se combinan las posiciones $(0, 1)$ y $(2, 3)$, que corresponden a los pares originales $(x(0), x(2))$ y $(x(1), x(3))$. En la segunda etapa, se combinan resultados intermedios separados por dos posiciones, por lo que $d_2 = 2$: se combinan las posiciones $(0, 2)$ y $(1, 3)$ del arreglo intermedio.

Por tanto, en una FFT de N puntos, la distancia entre posiciones sigue la secuencia $1, 2, 4, \dots, N/2$, mientras que el tamaño de los bloques sigue la secuencia $2, 4, 8, \dots, N$. La distancia y el tamaño de bloque están relacionados por $d_s = 2^{s-1}$ y $B_s = 2^s$, respectivamente.

Por lo que, para una FFT de $N = 1024$ puntos, se tiene:

- Número de etapas: $m = \log_2(1024) = 10$
- Tamaño de los bloques en cada etapa: $2, 4, 8, 16, 32, 64, 128, 256, 512, 1024$
- Distancia entre posiciones en cada etapa: $1, 2, 4, 8, 16, 32, 64, 128, 256, 512$

5. Ordenamiento de datos

En la sección anterior se ha descrito cómo se combinan los datos en cada etapa de la FFT. Sin embargo, para que el algoritmo funcione correctamente, es necesario reordenar los datos de entrada antes de iniciar las operaciones tipo mariposa.

Dado que el algoritmo de la FFT divide la señal en partes pares e impares, el orden de los datos de entrada debe ser reorganizado para que las posiciones de los elementos coincidan con el patrón de combinación de las mariposas. Este reordenamiento consta de dividir los índices en pares e impares, esto de manera recursiva hasta llegar a un caso base, y luego combinar los resultados de manera que se obtenga el orden correcto para las operaciones tipo mariposa.

Sin embargo, una forma más eficiente de realizar este reordenamiento es mediante la inversión de bits. En este método, cada índice de la señal de entrada se representa en binario y luego se invierten los bits para obtener el nuevo índice. Por ejemplo, para $N = 4$, los índices en binario son:

- 0 en binario: $00 \rightarrow$ invertido: $00 \rightarrow$ nuevo índice: 0
- 1 en binario: $01 \rightarrow$ invertido: $10 \rightarrow$ nuevo índice: 2
- 2 en binario: $10 \rightarrow$ invertido: $01 \rightarrow$ nuevo índice: 1
- 3 en binario: $11 \rightarrow$ invertido: $11 \rightarrow$ nuevo índice: 3

A nivel de código esto se puede implementar de diferentes formas, se puede implementar la inversión de bits por medio de cadenas de bits, o bien mediante operaciones de desplazamiento



y enmascaramiento.

Recordando que este algoritmo es deseable la mayor eficiencia posible, se opta por la implementación mediante operaciones de desplazamiento y enmascaramiento, ya que estas operaciones son más rápidas y eficientes que las manipulaciones de cadenas de bits.

A nivel de código la implementación fue la siguiente:

Listing 1: Implementación de la inversión de bits mediante operaciones de desplazamiento y enmascaramiento.

```
1 def inversion_bits(x):
2     N = len(x)
3     num_bits = int(np.log2(N))
4     x_invertido = np.zeros(N, dtype=complex)
5
6     for i in range(N):
7         indice_invertido = 0
8         for j in range(num_bits):
9             if (i >> j) & 1:
10                 indice_invertido |= 1 << (num_bits - 1 - j)
11             x_invertido[indice_invertido] = x[i]
12
13     return x_invertido
```

El funcionamiento del código consiste en recorrer cada índice de la señal, invertir el orden de sus bits y utilizar el índice invertido como nueva posición del dato. Para que este procedimiento sea válido, se supone que N es una potencia de dos, de modo que todos los índices pueden representarse utilizando el mismo número de bits.

En primer lugar, se obtiene el número de muestras y el número de bits necesarios para representar sus índices:

- $N = \text{len}(x)$ obtiene la longitud de la señal.
- $\text{num_bits} = \text{int}(\text{np.log2}(N))$ calcula el número de bits. Por ejemplo, si $N = 4$, entonces $\log_2(4) = 2$, por lo que los índices se representan con dos bits.
- $\text{x_invertido} = \text{np.zeros}(N, \text{dtype=complex})$ reserva un arreglo de salida de tamaño N . El tipo complejo permite conservar tanto entradas reales como resultados complejos de la FFT.

El ciclo externo recorre todos los índices originales:

```
1 for i in range(N):
2     indice_invertido = 0
```

Para cada índice i , la variable `indice_invertido` comienza en cero y se construye bit a bit.



El ciclo interno recorre cada posición binaria del índice:

```
1 for j in range(num_bits):
2     if (i >> j) & 1:
3         indice_invertido |= 1 << (num_bits - 1 - j)
```

Las operaciones utilizadas tienen el siguiente significado:

- $i \gg j$ desplaza los bits de i j posiciones a la derecha. Esto coloca en la última posición el bit que se desea analizar.
- $(i \gg j) \& 1$ extrae ese bit. El operador $\&$ es un AND bit a bit; al compararlo con 1, el resultado es 1 si el bit analizado estaba encendido y 0 si estaba apagado.
- $1 \ll (\text{num_bits} - 1 - j)$ crea una máscara con un 1 en la posición invertida correspondiente.
- $|\equiv$ combina esa máscara con `indice_invertido` mediante un OR bit a bit, activando el bit correspondiente sin modificar los bits que ya se habían construido.

Por ejemplo, para $N = 4$, el índice $i = 1$ se representa como 01_2 . Al invertir sus bits se obtiene $10_2 = 2$:

$$01_2 \longrightarrow 10_2$$

El código realiza esta asignación mediante:

```
1 x_invertido[indice_invertido] = x[i]
```

Por tanto, el valor que originalmente estaba en `x[1]` se almacena en `x_invertido[2]`. Para los cuatro índices de $N = 4$, el reordenamiento es:

Índice original	Representación binaria	Índice invertido
0	00_2	0
1	01_2	2
2	10_2	1
3	11_2	3

De forma que para una señal de entrada $x[n]$ de longitud $N = 1024$ el ordenamiento de los datos se realiza de manera similar, invirtiendo los 10 bits de cada índice para obtener la nueva posición de cada muestra.

6. Diagrama o esquema

Retomando los datos necesarios para la implementación de la FFT de $N = 1024$ puntos, se tiene:

- Número de etapas: $m = \log_2(1024) = 10$
- Tamaño de los bloques en cada etapa: 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024
- Distancia entre posiciones en cada etapa: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512

Se tiene el siguiente diagrama que representa la estructura general de la FFT de $N = 1024$ puntos:

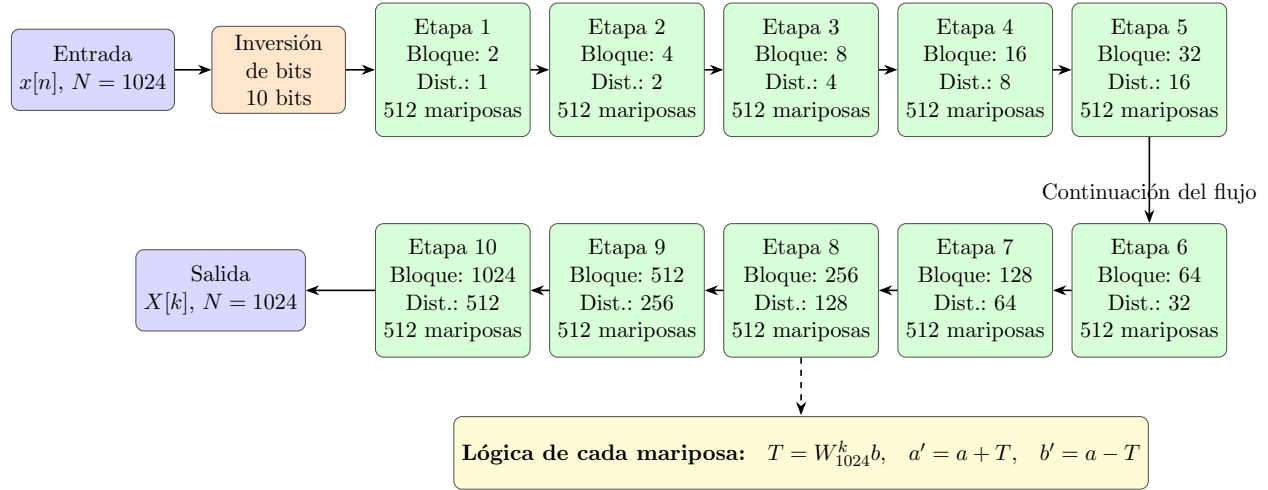


Figura 3: Representación general de una FFT de 1024 puntos. Cada etapa contiene $N/2 = 512$ mariposas

En la figura, cada rectángulo representa una etapa completa y no una sola mariposa. Por ello, no se dibujan las 512 conexiones individuales de cada etapa. La distancia indicada corresponde a la separación entre las dos posiciones que se combinan dentro de cada mariposa: comienza en 1 y se duplica hasta llegar a 512. El tamaño del bloque también se duplica, desde 2 hasta 1024, mientras que el número de mariposas permanece constante en $N/2 = 512$ por etapa.

7. Complejidad Computacional

La DFT directa tiene una complejidad temporal de $O(N^2)$, porque para calcular cada una de las N salidas debe combinarse toda la señal de entrada. La FFT Radix-2 aprovecha la descomposición en mariposas para reducir esta complejidad a $O(N \log_2 N)$. La lógica completa de la implementación desarrollada es la siguiente:

1. **Verificación del tamaño.** La función `verificacion_tamano` comprueba si la longitud de la señal es una potencia de dos. Si la longitud original es M y no cumple esta condición, se calcula la siguiente potencia de dos:

$$N' = 2^{\lceil \log_2(M) \rceil}$$



y se agregan $N' - M$ ceros al final de la señal. Este procedimiento se conoce como *zero-padding*. No forma parte de la FFT Radix-2 como tal; se incorpora para permitir que la función reciba señales de cualquier longitud.

2. **Inversión de bits.** La señal se reorganiza mediante `inversion_bits`. Para un tamaño $N = 2^m$, cada índice se representa con m bits y se invierte su orden. Este reordenamiento permite que las parejas que deben combinarse aparezcan en las posiciones adecuadas para ejecutar las mariposas de forma iterativa.
3. **Tabla de factores de giro.** Después del reordenamiento se construye el arreglo `W` con $N/2$ factores:

$$W_N^k = e^{-j\frac{2\pi}{N}k}, \quad 0 \leq k < N/2$$

Estos factores se calculan una sola vez y se reutilizan en las diferentes etapas. En cada etapa se seleccionan los factores necesarios mediante `salto`, evitando recalcular exponenciales complejas dentro de los ciclos de las mariposas.

4. **Recorrido de las etapas.** El ciclo `while` comienza con `tamano_bloque = 2` y duplica este valor hasta alcanzar N . Por tanto, se ejecutan:

$$m = \log_2(N)$$

etapas. El tamaño del bloque en la etapa s es $B_s = 2^s$, y la distancia entre los nodos superior e inferior de una mariposa es $B_s/2$.

5. **Recorrido de bloques y mariposas.** Para cada etapa, `range(0, N, tamaño_bloque)` localiza el inicio de cada bloque. Dentro de cada bloque, `range(mitad_bloque)` recorre sus mariposas. En cada operación se calculan los índices:

$$i_{sup} = inicio_bloque + mariposa, \quad i_{inf} = i_{sup} + mitad_bloque$$

6. **Operación in-place.** El factor seleccionado se multiplica por el nodo inferior para formar el término intermedio:

$$T = \omega x[i_{inf}]$$

Después se calculan la suma y la resta:

$$x[i_{sup}] = x[i_{sup}] + T, \quad x[i_{inf}] = x[i_{sup, original}] - T$$

En el código, `nodo_superior` y `nodo_inferior` guardan temporalmente ambos resultados para no perder el valor original del nodo superior antes de actualizar el arreglo. De esta manera, la siguiente mariposa y la siguiente etapa utilizan los resultados ya calculados sin requerir un segundo arreglo de trabajo.

7. **Resultado.** Cuando termina la última etapa, el arreglo contiene la FFT en el orden natural y la función devuelve `x`.



Retomando el algoritmo completo, la implementación considera la verificación opcional del tamaño, la inversión de bits, el cálculo de los factores de giro, el recorrido de las etapas y bloques, y la operación tipo mariposa in-place. El código correspondiente es:

Listing 2: Implementación de la FFT

```
1 def fft(x:np.ndarray) -> np.ndarray:
2     # Ordenamiento por inversión de bits
3     x = verificacion_tamano(x)
4     # Inversión de bits
5     x = inversion_bits(x)
6     N = len(x)
7
8     # Factores W
9     W = np.exp(-1j * (2 * np.pi / N) * np.arange(N//2))
10
11     # Tamaño de bloque inicial
12     tamano_bloque = 2
13     while tamano_bloque <= N:
14         mitad_bloque = tamano_bloque // 2
15
16         salto = N // tamano_bloque
17
18         w = W[0 : mitad_bloque * salto : salto]
19
20
21     for inicio_bloque in range(0, N, tamano_bloque):
22         for mariposa in range(mitad_bloque):
23             # Obtencion de los indices de los nodos superior e
24             # inferior
25             indice_superior = inicio_bloque + mariposa
26             indice_inferior = indice_superior + mitad_bloque
27
28             # Obtención del W
29             omega = w[mariposa]
30
31             # Resolucion de Mariposas
32             termino_intermedio = omega * x[indice_inferior]
33             nodo_superior = x[indice_superior] +
34                 termino_intermedio
35             nodo_inferior = x[indice_superior] -
36                 termino_intermedio
37
38             # Actualización de los nodos en el arreglo
39             x[indice_superior] = nodo_superior
40             x[indice_inferior] = nodo_inferior
```

```
38         tamaño_bloque *= 2
39
40
41     return x
```

Listing 3: Implementación de la verificación de tamaño

```
1 def verificacion_tamano(x):
2     N = len(x)
3     n = np.log2(N)
4
5     if not n.is_integer():
6         n = int(np.ceil(n))
7         N_nuevo = 2 ** n
8         x = np.pad(x, (0, N_nuevo - N), mode='constant')
9     return x
```

La complejidad temporal de la FFT se obtiene considerando sus componentes. La inversión de bits implementada recorre N índices y analiza $\log_2(N)$ bits por índice, por lo que cuesta $O(N \log N)$. La tabla de factores requiere $O(N)$ operaciones. En la parte principal existen $\log_2(N)$ etapas y en cada una se realizan exactamente $N/2$ mariposas; por tanto, el procesamiento de mariposas cuesta:

$$\frac{N}{2} \log_2(N) = O(N \log N)$$

En consecuencia, la complejidad temporal total de esta implementación, cuando la entrada ya tiene un tamaño potencia de dos, es:

$$T(N) = O(N \log_2 N)$$

La verificación de tamaño y el *zero-padding* son pasos adicionales y no pertenecen al núcleo de la FFT. Si la señal tiene longitud M y se rellena hasta $N' = 2^{\lceil \log_2(M) \rceil}$, el costo total pasa a expresarse como:

$$T(M) = O((N') \log(N'))$$

además del costo $O(N' - M)$ de crear el relleno. Por ello, cuando M no es potencia de dos, el algoritmo procesa más muestras que las originales y puede aumentar el tiempo y la memoria utilizados. Si M ya es una potencia de dos, entonces $N' = M$, no se agregan ceros y se conserva la complejidad habitual $O(M \log_2 M)$.



8. Conclusiones

La Transformada de Fourier es una herramienta matemática fundamental en el procesamiento de señales digitales, ya que permite analizar la frecuencia de las señales, lo cual tiene múltiples aplicaciones en áreas como telecomunicaciones, audio, imagen y radar. Dado que para computar una señal se debe discretizar y muestrear, la FFT es una herramienta esencial para convertir señales del dominio del tiempo al dominio de la frecuencia de manera eficiente.

La FFT es un algoritmo eficiente y fácil de implementar, ya que se reduce a ciclos iterativos de operaciones tipo mariposa, que consisten en sumas, restas y multiplicaciones por factores de giro. Comprender este algoritmo es de suma importancia para cualquier ingeniero ya que esta implícito en el día a día de la ingeniería, siendo una herramienta fundamental para el análisis de señales y sistemas.

Por otro lado, la FFT fue implementada mediante software, lo cual fue muy viable, ya que la implementación no depende de un lenguaje de programación específico, sino que se puede adaptar a cualquier lenguaje que soporte operaciones matemáticas y estructuras de datos, incluso si se busca una eficiencia mayor se puede llegar a implementar en bajo nivel, como en ensamblador, para aprovechar al máximo los recursos de hardware. Por otro es posible implementar la FFT en hardware, sin embargo en una primer instancia no es lo más viable, ya que requiere de un conocimiento más profundo de la arquitectura del hardware así como más tiempo y recursos para su implementación. Por ello, la implementación en software es más recomendable, ya que permite una mayor flexibilidad y facilidad de uso.

Referencias

- Burrus, S. (2022, 1 de noviembre). *8.5: Evaluación de los algoritmos FFT de Cooley-Tukey*. [https://espanol.libretexts.org/Bookshelves/Ingenieria/Transformadas_Rapidas_de_Fourier_\(Burrus\)/08%3A_El_algoritmo_de_transformada_r%C3%A1pida_de_Fourier_Cooley-Tukey/8.05%3A_Evaluaci%C3%B3n_de_los_algoritmos_FFT_de_Cooley-Tukey](https://espanol.libretexts.org/Bookshelves/Ingenieria/Transformadas_Rapidas_de_Fourier_(Burrus)/08%3A_El_algoritmo_de_transformada_r%C3%A1pida_de_Fourier_Cooley-Tukey/8.05%3A_Evaluaci%C3%B3n_de_los_algoritmos_FFT_de_Cooley-Tukey)
- NumPy Developers. (2026). *Discrete Fourier Transform*. <https://numpy.org/doc/2.5/reference/routines.fft.html#>
- Oppenheim, A., & Schaffer, R. (2011). *Tratamiento de señales en tiempo discreto* (3.^a ed.). Pearson.
- Proakis, J., & Manolakis, D. (2010, 1 de enero). *Tratamiento digital de señales* (4.^a ed.). PRENTICE HALL/PEARSON.